

Forge Extensions

Invariant ideals, finite covers,
constructive integer projection,
and proof-directed synthesis

Prepared for Vladimir Reshetnikov
15 September 2026

Abstract

Forge already has a broad design for proof planning above Lean’s `grind`. This article develops additional algorithms rather than another description of that architecture. The central addition computes the greatest finite-dimensional space of invariant generators whose transition pullbacks admit degree-bounded polynomial multipliers. A descending linear-algebra calculation replaces the apparently bilinear simultaneous search for generators and multipliers. Two further workers prove properties of all finite recursively constructed trees by finite-algebra covers, and eliminate one existential integer variable while constructing a symbolic witness without modulus-sized residue tables. A fourth, unimplemented integration proposal decomposes higher-order synthesis into proof-directed fold contracts, motivated by a specific open composition problem in `Leant` and `Djex`.

The archive contains exact standard-library Python implementations, checks of 51 constant-multiplier packets, 14 polynomial-multiplier packets, 136 finite-algebra certificates, and 221 symbolic integer projections. It also contains 21 passing test methods, a differential integer experiment with 3400 concrete parameter valuations, and explicit incompleteness and corruption controls. These are Python-checked research artifacts, not Lean kernel proofs or a measured improvement over an existing tactic. The mathematical soundness and fragment-completeness claims are proved in the text; the remaining Lean implementation work is specified separately.

Contents

1	The delta: what this article adds	3
1.1	The starting point is already ambitious	3
1.2	A common problem, not another common framework	3
1.3	Evidence and scope	4
2	From conserved quantities to invariant packets	4
2.1	Preservation of a zero set is weaker than conservation	4
2.2	Compute the entire admissible space	5
2.3	The actual linear system	5
2.4	A nonlinear case the constant packet still finds	6
2.5	An explicit limitation, not a reason to abandon the method	6
3	Invariant ideals with bounded multipliers	6
3.1	A monotone relaxation of the packet condition	7
3.2	Why this does not solve a bilinear system	7
3.3	Worked nonlinear certificates	8
3.4	What the checker actually certifies	9
3.5	Cost, refinement, and a concrete production route	9
3.6	Extension to control-flow locations	9
4	Finite-algebra covers for recursive objects	10
4.1	Replace unbounded syntax by the finite summary the goal needs	10
4.2	Positive certificates are closed sets, not search logs	10
4.3	Search and a genuine counterexample	11
4.4	Example: two related counts in all binary trees	11
4.5	Example: equality of folds without assuming associativity	12
4.6	The source-level bridge is a proved commuting diagram	12
4.7	How this becomes a Forge worker	13
5	Constructive integer projection without residue tables	13
5.1	The exact supported fragment	13
5.2	Normalize arbitrary fixed coefficients in congruences	13
5.3	Merge congruences with non-coprime moduli	14
5.4	Convert inequalities into an interval	14
5.5	Choose the nearest congruent point	15
5.6	A witness program, not an exponentially large normal form	15
5.7	How to discharge a quantified Lean goal	16
5.8	The remaining verification work	16
5.9	Limits that must survive the interface	16
6	Proof-directed higher-order sketch decomposition	17
6.1	The specific obstruction in Leant and Djex	17
6.2	The proof rule should determine the holes	17
6.3	Indexing explains why the carrier must be a function	17
6.4	Separate executable context from specification context	18

6.5	A concrete search protocol	18
6.6	Church encodings require an explicit representation law	19
6.7	What should be measured in the real integration	20
7	A cross-worker proof, with a precise division of labor	20
8	Lean implementation plan and concrete dependencies	20
8.1	Use the existing outer boundary	20
8.2	A minimal certificate-to-proof path for each worker	21
8.3	Library choices, with their roles kept narrow	22
8.4	Mathematical contracts for the native interface	22
8.5	One particularly useful proof-reuse boundary	22
9	Executed experiments and what they establish	23
9.1	Environment and units	23
9.2	Invariant tests and ablations	23
9.3	Finite-algebra differential checks	24
9.4	Integer differential checks	24
9.5	Corruption and implementation controls	24
9.6	Timings are diagnostic only	25
10	Implementation order and acceptance gates	25
10.1	First: one source theorem per certificate family	25
10.2	Second: native certificate checkers, not native searches	25
10.3	Third: benchmark the actual new capability	25
10.4	Fourth: test the original higher-order query unchanged	26
11	Assessment	26
A	Reproduction and source revisions	26
A.1	Archive layout	26
A.2	Commands	26
A.3	Pinned audit references	27
B	Certificate schemas and small API examples	27
B.1	Exact polynomials	27
B.2	Invariant packets	28
B.3	Finite algebras	28
B.4	Integer projection	29
C	Acceptance ledger	29

1 The delta: what this article adds

1.1 The starting point is already ambitious

At the audited revision, Forge’s merged article already specifies an outer proof-planning layer, demand-directed theorem use, induction and accumulator generalization, lemma discovery, arithmetic witness synthesis, exact algebraic certificates, and a carefully scoped verification boundary. Its structural chapter implements polynomial recurrence discovery and conserved quantities; it explicitly identifies more general simultaneous invariant preservation as a next step. The repository also implements finite residue tables for integer witnesses. Those are the immediate baselines for this article, not capabilities being proposed again [1, 2, 5].

The Leant/Djex review already incorporated into Forge is substantial. It covers source-owned evidence, negative-verdict restrictions, kernel-checking options, process boundaries, neutral identities, and budget accounting. Repeating that material would not make the tactic more powerful [3]. Here the additions are concrete, separately checkable inference mechanisms.

Area	Audited baseline	Addition developed here
Polynomial invariants	Conserved quantities; general multiplier identities suggested but not synthesized	A greatest-fixed-point algorithm for simultaneous generators with bounded polynomial multipliers
Recursive finite summaries	General structural induction and lemma search	Finite-algebra closure certificates and concrete counterexample trees
Integer witnesses	Lattice certificates, affine templates, finite residue tables	Exact one-output projection with a straight-line floor/modulus witness and feasibility guard
Higher-order synthesis	Typed candidate search; a documented composition miss	Contract-first fold sketches, executable/ghost context separation, and reusable hole theorems

The first three rows have executable prototypes in the archive. The fourth is a detailed integration design and two hand-written Lean specimens; it is not a claim that the original Leant query has been solved. Repository searches for `pullback`, `automata`, and `Cooper` returned no matches, but that is not an exhaustive proof of absence from every frozen proposal. The novelty claim is a concrete extension of the reviewed merged design and implementation, not invention of tree automata, Presburger arithmetic, or abstract interpretation.

1.2 A common problem, not another common framework

The new workers address three different reasons local proof search gets stuck. A true equality may need to be proved *together with* other equalities that change under the transition. A recursive object may be unbounded while its relevant semantic summary is finite. An existential integer may require a piecewise, periodic function of parameters rather than a rational affine term. The higher-order proposal addresses a fourth issue: a correct component can exist in the search space yet be buried by irrelevant same-typed alternatives.

Each worker changes the mathematical object searched for. It does not merely increase a timeout or select a larger theorem collection. Successful output is, respectively, an invariant packet, an inductive cover or concrete tree, and a witness program with a guard. Their source-level meanings are distinct and must remain distinct in Forge’s result types.

1.3 Evidence and scope

All experiments in this archive ran in one environment and are reproducible with Python’s standard library. The checkers use exact integers and rational numbers. They do not use numerical tolerances. Nevertheless, shared arithmetic code and shared certificate elaboration remain possible common-mode failure sources. The archive has no Lean executable available, no implementation of the `forge` tactic, and no benchmark against `grind`, `Aesop`, or another Lean automation tool. The optional Lean files are explicitly marked NOT RUN. These distinctions are essential when interpreting the tables in Section 9.

2 From conserved quantities to invariant packets

Status. Implemented in `invariant_space.py`; exact identity checker and recorded experiments included.

2.1 Preservation of a zero set is weaker than conservation

Let a state be $x \in \mathbb{Q}^n$ and let $T_1, \dots, T_r : \mathbb{Q}^n \rightarrow \mathbb{Q}^n$ be polynomial transitions. Every transition is allowed at every state in the initial fragment. Let the initial states be the union of the images of polynomial maps $\sigma_1, \dots, \sigma_s$. Parameters of these maps range over all rational values. The implementation pads parameter maps with unused variables so all polynomials use one common arity.

A conserved polynomial satisfies $p(T_a(x)) = p(x)$ everywhere. An invariant equality only requires that $p = 0$ remain true on reachable states. Requiring conservation is therefore unnecessarily strong. With several polynomials, a sufficient preservation condition is

$$P(\sigma_b(u)) = 0, \quad P(T_a(x)) = M_a P(x), \quad (1)$$

where $P = (p_1, \dots, p_k)^\top$ and M_a is a rational matrix. No individual p_i need be conserved, and M_a need not be invertible, positive, or stable in any analytic sense.

Theorem 2.1 (Packet soundness). *If the polynomial identities in (1) hold, then $p_i(x) = 0$ for every reachable state x and every i .*

Proof. The initial identities establish the claim at a state $\sigma_b(u)$. If $P(x) = 0$ and the next transition is T_a , then $P(T_a(x)) = M_a P(x) = M_a 0 = 0$. Induction on the length of a finite execution gives the result. No termination statement about an infinite execution is required. $\square$

Example 2.2 (A packet that conservation misses). Take

$$T(x, y, z) = (y, z, x), \quad (x_0, y_0, z_0) = (t, t, t).$$

With $p = x - y$ and $q = y - z$,

$$\begin{pmatrix} p \circ T \\ q \circ T \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & -1 \end{pmatrix} \begin{pmatrix} p \\ q \end{pmatrix}.$$

Thus $x = y = z$ is preserved. In contrast, a conserved linear polynomial has equal coefficients on x, y, z ; requiring that it vanish at every (t, t, t) forces those coefficients and the constant term to be zero. The conserved-linear subspace is zero, while the invariant-linear subspace has dimension two. The recorded prototype returns the equivalent basis $(z - x, y - x)$, not the hand-chosen basis displayed above.

This is a strict increase in what the *selected template method* can certify. It is not a demonstration that Lean’s existing tactics cannot prove this elementary example with suitable induction and hypotheses.

2.2 Compute the entire admissible space

Fix a finite-dimensional template space

$$V_d = \{p \in \mathbb{Q}[X_1, \dots, X_n] : \deg p \leq d\}, \quad N = \dim V_d = \binom{n+d}{d}.$$

A sparse template can replace all monomials, but the full degree template has a particularly clean completeness statement. Polynomial pullback $T_a^* : p \mapsto p \circ T_a$ is a *linear operator in p* , even when T_a is nonlinear. Its output may have a higher degree; that is not a problem for exact coefficient comparison.

First compute

$$W_0 = \{p \in V_d : p \circ \sigma_b = 0 \text{ as a polynomial for every } b\}.$$

Substitution of each template monomial into each initial map gives a matrix of rational coefficients. Its kernel is W_0 . This is not interpolation over a sample of initial states.

For a current space W , apply the restriction

$$W \longleftarrow W \cap (T_a^*)^{-1}(W). \quad (2)$$

Cycle through all transitions until a full pass makes no dimension decrease. Because every update only removes vectors, equality of dimensions implies equality of spaces; comparing pretty-printed bases is unnecessary.

2.3 The actual linear system

Let $b_1, \dots, b_k$ be a basis of the current W . Form coefficient columns for

$$b_1 \circ T_a, \dots, b_k \circ T_a, -b_1, \dots, -b_k.$$

A null vector (c, h) says

$$\sum_i c_i (b_i \circ T_a) = \sum_j h_j b_j.$$

Project a basis of this kernel to its first k coordinates and take the span of the resulting polynomials $\sum_i c_i b_i$. This is exactly the new space in (2). Kernel vectors with $c = 0$ contribute no polynomial and disappear during row reduction.

```

W := kernel(initial_substitution_constraints(V_d))
repeat:
  old_dimension := dimension(W)
  for T in transitions:
    A := coefficient_columns(pullback(T, basis(W)))
    B := coefficient_columns(basis(W))
    K := nullspace([A | -B])
    W := span(project_left(K) * basis(W))
until dimension(W) = old_dimension
solve coordinates of each pullback in the final basis
return basis(W), transition_matrices

```

The source uses rational row reduction, deterministic monomial order, and canonical rational coefficients. It is deliberately small rather than an optimized symbolic linear-algebra library.

Theorem 2.3 (Greatest pullback-stable subspace). *The restriction algorithm terminates. Its result is the greatest subspace $W \subseteq W_0$ for which $T_a^* W \subseteq W$ for every transition a . There are at most N strict dimension decreases.*

Proof. Every update is an intersection with a linear preimage, hence is a subspace of the previous space. Strict inclusion decreases dimension. A full pass with no decrease leaves every restriction unchanged, establishing closure. If $U \subseteq W_0$ is any common pullback-stable subspace, then $U \subseteq W$ before an update implies $T_a^*U \subseteq U \subseteq W$, and hence $U \subseteq W \cap (T_a^*)^{-1}(W)$ afterwards. Induction over updates shows U is contained in the final space. $\square$

Theorem 2.4 (Exact degree-bounded equality invariants for affine transitions). *If every transition is affine, the final space equals the set of all polynomials of degree at most d that vanish on the reachable states defined above.*

Proof. Let E be that set. It is a subspace of W_0 . For $p \in E$, an affine substitution preserves the degree bound, and $p \circ T_a$ vanishes on every reachable state because each successor is reachable. Thus $T_a^*E \subseteq E$, so Theorem 2.3 gives $E \subseteq W$. Conversely the final basis has a packet certificate, and Theorem 2.1 gives $W \subseteq E$. $\square$

This is a lifted, degree-bounded linear-invariant computation, in the lineage of Karr-style affine relation analysis. It is not the much more general algebraic algorithm for all polynomial invariants of affine programs studied by Hrushovski, Ouaknine, Pouly, and Worrell [10, 11].

2.4 A nonlinear case the constant packet still finds

Consider $T(n, s) = (n + 1, s + n^2)$ with initial state $(0, 0)$. In degree three the successive recorded dimensions are

$$9 \longrightarrow 5 \longrightarrow 3 \longrightarrow 2 \longrightarrow 1 \longrightarrow 1.$$

The surviving basis polynomial is

$$p(n, s) = s - \frac{n}{6} + \frac{n^2}{2} - \frac{n^3}{3}.$$

Its checked pullback is $p \circ T = p$. Clearing denominators yields $6s = n(n - 1)(2n - 1)$. Forge’s existing recurrence worker also targets this kind of example; it is included here to test compatibility with a known successful fragment, not as a new capability claim.

2.5 An explicit limitation, not a reason to abandon the method

For $T(x, y) = (x^2, y^2)$ and initial states (t, t) , the true linear invariant is $x - y = 0$. But

$$(x - y) \circ T = (x + y)(x - y),$$

which is not in the one-dimensional span of $x - y$. At $d = 1$ the constant-packet algorithm returns the zero space. The failure comes from its certificate language, not from the absence of a linear invariant. The next section removes this particular obstruction without a bilinear solver.

3 Invariant ideals with bounded multipliers

Status. Implemented in `ideal_space.py`. Polynomial multiplier certificates and negative controls are included.

3.1 A monotone relaxation of the packet condition

For a nonnegative multiplier-degree bound e , define

$$J_e(W) = \text{span}_{\mathbb{Q}}\{mp : p \in W, m \text{ a monomial, } \deg m \leq e\}. \quad (3)$$

This is a finite-dimensional *truncated multiple space*, not necessarily an ideal: it need not be closed under further multiplication. In terms of a basis $p_1, \dots, p_k$ of W , it consists precisely of sums $\sum_j h_j p_j$ with $\deg h_j \leq e$. The notation J_e is convenient, but using the full ideal's closure properties here would be a mistake.

Replace the restriction in the previous section by

$$F_{a,e}(W) = W \cap (T_a^*)^{-1}(J_e(W)). \quad (4)$$

This operation is monotone: $U \subseteq W$ implies $J_e(U) \subseteq J_e(W)$, and therefore $F_{a,e}(U) \subseteq F_{a,e}(W)$. It is also deflationary, since $F_{a,e}(W) \subseteq W$.

The final certificate becomes

$$p_i \circ \sigma_b = 0, \quad p_i \circ T_a = \sum_{j=1}^k h_{aij} p_j, \quad \deg h_{aij} \leq e. \quad (5)$$

The same induction proof as before establishes soundness, because every term on the right vanishes when all p_j vanish. Coefficients may be polynomials in the current state. They do not have to be invariant or bounded in magnitude.

3.2 Why this does not solve a bilinear system

Forge correctly notes that writing unknown generators p_i and unknown multipliers h_{ij} simultaneously produces bilinear coefficient equations. The algorithm here does not solve that system directly. It starts with *all* initially valid polynomials in a fixed finite template. At every restriction step the current generator basis is already known. Consequently, all products of that basis with bounded monomials are known polynomials, and finding which pullbacks belong to their span is ordinary linear algebra.

For current basis $b_1, \dots, b_k$ and multiplier monomials $m_1, \dots, m_K$, form columns

$$b_i \circ T_a \quad (1 \leq i \leq k), \quad -m_\ell b_j \quad (1 \leq j \leq k, 1 \leq \ell \leq K).$$

Take the kernel and project to the first k coordinates as before. This computes (4) exactly. After convergence, solve for the coordinates of each $b_i \circ T_a$ in the final product list and regroup the coefficients into polynomials h_{aij} .

```

W := all initially-zero polynomials in V_d
repeat:
  old_dimension := dimension(W)
  for T in transitions:
    products := [m * p for p in basis(W), deg(m) <= e]
    columns := pullback(T, basis(W)) ++ (-products)
    K := nullspace(coefficient_matrix(columns))
    W := span(left_projection(K) * basis(W))
until dimension(W) = old_dimension
H := coordinates(pullbacks(final_basis), final_products)
return final_basis, regroup_as_polynomial_matrices(H)

```

This is a change in the search problem, not a linearization that drops products of unknown quantities and hopes they can be repaired later. Every certificate identity is checked with all products present.

Theorem 3.1 (Greatest bounded-multiplier generator space). *The algorithm terminates and returns the greatest subspace $W \subseteq W_0$ such that $T_a^*W \subseteq J_e(W)$ for every a .*

Proof. The sequence descends inside finite-dimensional V_d , so it stabilizes. Stability of a complete transition pass is exactly the required inclusion for each transition. Let $U \subseteq W_0$ satisfy the same inclusions. If $U \subseteq W$, then $T_a^*U \subseteq J_e(U) \subseteq J_e(W)$, so $U \subseteq F_{a,e}(W)$. The induction over updates proves greatestness. $\square$

Corollary 3.2 (Relative completeness of discovery). *Any finite collection of degree-at-most- d polynomials that vanishes initially and admits preservation identities with multiplier degrees at most e spans a subspace of the computed result. In particular, the search does not miss a certificate packet in this fixed fragment merely because its generators and multipliers would have appeared bilinearly in a direct ansatz.*

Proof. The span of the supplied collection is initially zero. Every linear combination of its pullbacks is a bounded-degree polynomial combination of the same collection, so its span satisfies the hypothesis of Theorem 3.1. $\square$

The corollary is about *packets closed within the selected template and multiplier bound*. It is not a decision procedure for all invariants, all polynomial implications, or all true goals. A valid invariant may require auxiliary generators above degree d , or preservation multipliers above degree e , or a different certificate such as a radical-ideal argument.

3.3 Worked nonlinear certificates

For squaring on the diagonal, degree $d = 1$, $e = 0$ fails and $e = 1$ succeeds:

$$p = y - x, \quad p(x^2, y^2) = (x + y)p(x, y).$$

More generally, for $T_k(x, y) = (x^k, y^k)$,

$$(y - x) \circ T_k = \left(\sum_{j=0}^{k-1} x^j y^{k-1-j} \right) (y - x).$$

The archive synthesizes these packets for $k = 2, \dots, 9$, using $e = k - 1$.

An initial union is handled without choosing one component. Let initial states be $(t, 0)$ or $(0, t)$ and use the same squaring transition. In degree two, $W_0 = \text{span}\{xy\}$. Then

$$(xy) \circ T = x^2 y^2 = (xy)(xy).$$

The degree-one multiplier limit fails, while $e = 2$ recovers the packet. The checker verifies both initial substitutions; checking only the first would silently change the subject.

For the parabola $\sigma(t) = (t, t^2)$, take $p = y - x^2$. Then

$$p \circ T = y^2 - x^4 = (y + x^2)(y - x^2).$$

Again $d = 2, e = 2$ succeeds. These examples exercise changing zero sets, unions of initial images, and nonlinear multipliers without numerical sampling.

3.4 What the checker actually certifies

The certificate contains the exact problem, generator polynomials, and one polynomial matrix per transition. Its checker verifies arities, canonical rational coefficients, degree limits, all initial substitutions, and every identity in (5). It calls neither nullspace computation nor invariant synthesis. The replay driver replaces those functions with routines that raise an exception if called.

An accepted certificate proves only the preservation claim represented by the packet. It does *not* certify that the packet is greatest. In particular, an empty packet is a vacuously valid preservation certificate, not independently checked evidence that no useful invariant exists. Certifying greatestness would require additional row-space or elimination evidence for every restriction step. Forge should not upgrade an empty returned packet to a mathematical refutation of its goal.

3.5 Cost, refinement, and a concrete production route

There are $N = \binom{n+d}{d}$ template monomials and $K = \binom{n+e}{e}$ multiplier monomials. At a stage with k generators, the restriction matrix has $k(1 + K)$ columns. If the largest transition degree is t , its rows range over monomials of degree at most $\max(dt, d + e)$, though sparsity can make the actual support much smaller. At most N strict restrictions occur, and a simple pass scheduler uses at most $r(N + 1)$ edge checks. None of these bounds controls coefficient bit growth; rational elimination and substitution expansion can dominate memory.

A useful initial scheduling order is $(d, e) = (1, 0), (1, 1), (2, 0), (2, 1), (2, 2)$, followed by problem-specific templates. This order is a proposed heuristic, not a measured optimum. Increasing e can recover certificates that a smaller bound cannot express. However, a larger-bound run must begin with the full new W_0 , not the old final subspace: using the old answer as the only starting space would prevent lost generators from reappearing.

Use sparse coefficient collection in the Lean-facing front end and an exact rational matrix backend for the proposer. `python-flint`'s `fmpq_mat` offers rational row reduction and solving as a concrete external option; the included implementation uses `fractions.Fraction` to remain self-contained [16]. In Lean, reify into a small polynomial syntax, prove its denotation correct, check coefficient identities, and instantiate a generic reachability-invariant theorem. Mathlib's multivariate-polynomial evaluation homomorphisms provide an appropriate semantic target [17]. Neither external library is needed to trust a returned packet once the Lean identity checker exists.

3.6 Extension to control-flow locations

A useful unimplemented next step is a family of generator spaces W_ℓ , one per control-flow location. Initial conditions restrict entry locations; a location without an initial state starts with its full template. For an edge $\ell \xrightarrow{T} k$, require

$$T^*W_k \subseteq J_e(W_\ell).$$

The corresponding restriction is on the *target* generator space:

$$W_k \leftarrow W_k \cap (T^*)^{-1}(J_e(W_\ell)).$$

This remains monotone on the product of the subspace lattices and terminates after at most the sum of their template dimensions in strict decreases. A worklist rechecks an edge when either its source or target space changes. The greatestness proof is the same containment induction as above.

An equality guard $g = 0$ can contribute bounded multiples of g to the right-hand side, giving identities of the form $p_k \circ T = \sum h_j p_{\ell_j} + qg$. Inequality guards do not vanish and cannot be

inserted into this equation as if they did. Ignoring such guards gives a sound over-approximation but may lose invariants; exploiting them requires a separate proved implication. This control-flow extension is specified here, not implemented in the single-location prototype.

4 Finite-algebra covers for recursive objects

Status. Implemented in `finite_cover.py`; deterministic algebras with constructor arities zero, one, and two; positive covers and concrete negative certificates.

4.1 Replace unbounded syntax by the finite summary the goal needs

A recursive datatype may contain arbitrarily large values while a property of interest depends on a finite summary. Examples include counts modulo a fixed integer, a finite-state parser’s final state, a bounded flag configuration, and equality of two folds into a finite algebra. Unrolling constructors to a fixed depth is not a proof over all values. A finite summary with proved constructor laws supplies a different route: compute an inductive cover of summary states.

Let Σ be a finite ranked signature. A constructor c has an arity $a(c)$, and $\mathcal{T}_\Sigma$ is the set of finite ground trees built from these constructors. A deterministic finite algebra consists of a finite set Q and functions

$$\delta_c : Q^{a(c)} \longrightarrow Q.$$

Every ground tree has a uniquely determined summary $\llbracket t \rrbracket$, computed from the leaves upwards. Given $G \subseteq Q$, the decision problem is

$$\forall t \in \mathcal{T}_\Sigma, \quad \llbracket t \rrbracket \in G. \quad (6)$$

This is a standard finite-tree-automata problem. The contribution to Forge is the certificate form, source-level bridge, and scheduling as an alternative to unbounded structural search. Lammich’s machine-checked tree-automata development provides a mature reference for the underlying algorithms and witness construction, albeit in Isabelle rather than Lean [12].

4.2 Positive certificates are closed sets, not search logs

Definition 4.1 (Inductive cover). A set $R \subseteq Q$ is an inductive cover if for every constructor c and every tuple $q_1, \dots, q_{a(c)} \in R$,

$$\delta_c(q_1, \dots, q_{a(c)}) \in R.$$

For a nullary constructor this requires its designated state to belong to R .

Theorem 4.2 (Finite-cover soundness). *If R is an inductive cover and $R \subseteq G$, then (6) holds.*

Proof. Induct on a finite constructor tree. The induction hypotheses put every child summary in R . The closure check for its constructor puts the parent summary in R . For nullary constructors this is exactly the required initial-state check. Finally use $R \subseteq G$. $\square$

The positive certificate can therefore be only a list of state identifiers. The checker verifies membership, distinctness, all constructor closures, and $R \subseteq G$. It does not need to trust how the proposer found R and does not need to recompute the least reachable set. A larger closed cover is just as sound as the least one, provided it stays inside G .

4.3 Search and a genuine counterexample

The proposer computes the least constructor-closed set by saturation. Every new state retains a witness tree node whose children refer to previously constructed witness nodes. If a state outside G is discovered, its witness is returned immediately. The witness is a directed acyclic graph that denotes an ordinary finite tree, possibly with shared subtrees.

```

reachable := empty state-to-witness map
repeat:
  for each constructor c:
    for each tuple of already reachable child states:
      q := delta_c(tuple)
      if q is new:
        record q with a node pointing to child witnesses
        if q not in good:
          return concrete_tree_DAG(q)
  if no new state was added:
    return inductive_cover(keys(reachable))
  if resource budget is exhausted:
    return unknown

```

The negative checker requires every child index to be strictly smaller than the index of its parent. It evaluates each node using the input algebra and checks that the designated root state is outside G . No search result tag is accepted as a substitute for this tree. A malformed self-reference is rejected even if its proposed summary state happens to be bad.

Theorem 4.3 (Completeness for the explicit finite algebra). *Without a resource cutoff, saturation either returns an inductive cover inside G or a finite tree whose summary is outside G .*

Proof. At most $|Q|$ new states can appear. Every stored state is realized by its recorded finite tree. Conversely, induction on tree height shows that the summary of every finite tree eventually enters the closure. Therefore the fixed point is exactly the set of realizable summaries. If it contains a bad state, the associated finite tree refutes the universal claim; otherwise it is a positive cover. $\square$

This completeness statement is for the supplied finite algebra. It is not a claim that arbitrary recursive functions admit a sufficiently precise finite abstraction or that finite-model reasoning decides all Lean inductive goals.

4.4 Example: two related counts in all binary trees

For a full binary tree let $L(t)$ be its number of leaves and $N(t)$ its number of internal nodes. Fix $m \geq 1$. Use state space

$$Q = (\mathbb{Z}/m\mathbb{Z})^2,$$

with leaf summary $(1, 0)$ and node transition

$$(\ell_1, n_1), (\ell_2, n_2) \mapsto (\ell_1 + \ell_2, n_1 + n_2 + 1).$$

The desired good set is

$$G = \{(\ell, n) : \ell - n \equiv 1 \pmod{m}\}.$$

The prototype discovers this set as its reachable cover. The rule closes because

$$(\ell_1 + \ell_2) - (n_1 + n_2 + 1) = (\ell_1 - n_1) + (\ell_2 - n_2) - 1 \equiv 1.$$

This is a theorem about *all* finite binary trees, not just the trees used to reach the m states during search.

At $m = 12$ the explicit product has 144 states and the binary transition table has $144^2 = 20,736$ entries. The recorded search reaches 12 states, evaluates 145 distinct constructor/child-state tuples, and visits 235 tuples including revisits. These are separate quantities. Building and validating the entire explicit transition table still costs work proportional to its 20,736 entries; 145 is not an end-to-end complexity claim. A native implementation can use a proved modular transition function instead of an explicit table, but then that function's connection to the source fold must be checked.

The finite certificate establishes the selected congruence. It does not justify upgrading that congruence to the integer equality $L = N + 1$ merely because the same equality motivated the choice of states.

4.5 Example: equality of folds without assuming associativity

Let $(A, \star)$ be an arbitrary finite magma: an operation on a finite set, with no associativity or identity assumption. A tree has leaf labels in A . Compare its fold with the fold of its mirror. The product summary is

$$(u, v) = (\text{fold}(t), \text{fold}(\text{mirror}(t))).$$

A leaf a has summary (a, a) . For child summaries $(u_1, v_1), (u_2, v_2)$, the node summary is

$$(u_1 \star u_2, v_2 \star v_1).$$

The good states are the diagonal pairs. Saturation either certifies that every reachable pair is diagonal or constructs an actual tree with unequal folds.

If $\star$ is commutative, the diagonal is closed; associativity is irrelevant. If some $a \star b \neq b \star a$, the tree with leaves a, b already provides a counterexample. The experiment includes both classes, as well as unrelated random algebras checked against a separately written closure oracle. This is a small but meaningful regression suite for product construction, constructor order, and witness reconstruction.

4.6 The source-level bridge is a proved commuting diagram

A finite table is not automatically a model of a Lean definition. For a source datatype D and summary $\alpha : D \rightarrow Q$, each constructor must have a proved compatibility theorem

$$\alpha(c_D(t_1, \dots, t_a)) = \delta_c(\alpha(t_1), \dots, \alpha(t_a)).$$

The final target must follow from $\alpha(t) \in G$. These bridge theorems are part of preparation. With them, Theorem 4.2 becomes a source proof through ordinary structural induction.

For an infinite payload type, such as integer-labelled trees, a finite classification of labels needs its own exhaustive coverage proof. A positive cover can safely over-approximate possible labels and transitions. A bad *abstract* state does not then automatically give a concrete source counterexample: its abstract labels or transitions might be unrealizable. Negative export requires reconstructing actual source constructors and values and checking the resulting tree. The current prototype uses a finite explicit signature and exact transition tables, so it avoids this extra abstraction layer.

A Lean implementation should begin with non-indexed datatypes and registered summary homomorphisms. Mutual recursion, dependent indices, quotient representations, coinductive streams, and general well-founded recursion need additional semantics and should not be silently flattened into this signature.

4.7 How this becomes a Forge worker

A preparation pass recognizes a goal about a fold into `Fin m`, a finite product, or another explicitly enumerated type. It searches the registered summary laws, constructs the relevant product algebra, and sends its pure finite description to the proposer. The returned cover is checked using `Finset/Fintype` data or an array-based reflected checker. The resulting theorem is then introduced as an ordinary local fact.

The first implementation does not need a general automata library. A worklist, a dense or sparse state table, and the small cover checker suffice. A more general library may become useful for nondeterministic summaries or language operations, but importing such a library does not replace the Lean source bridge. The reference formalization is useful as an algorithm and refinement model, not as automatically transferable Lean authority [12].

Use a budget in state discoveries, tuple evaluations, table size, and certificate size. The explicit prototype implements a transition-evaluation cutoff; it is not hardened against arbitrary hostile table sizes. State-product blow-up is the main structural limit. Goal-directed products should include only the summaries needed by the target, while any omission must still leave a sound abstraction.

5 Constructive integer projection without residue tables

Status. Implemented in `presburger.py`; exact symbolic witness programs, feasibility guards, Bezout receipts, and differential checks.

5.1 The exact supported fragment

Let $x = (x_1, \dots, x_n) \in \mathbb{Z}^n$ be parameters and let $y \in \mathbb{Z}$ be the output. The input is a conjunction

$$\Phi(x, y) \equiv \bigwedge_i a_i y \leq \beta_i(x) \quad \wedge \quad \bigwedge_j c_j y \equiv \delta_j(x) \pmod{m_j}, \quad (7)$$

where the coefficients a_i, c_j, m_j are fixed integers, $m_j > 0$, and the right sides are integer affine expressions in x . Coefficients may be negative or zero. An empty conjunction is allowed. There is exactly one existential output variable in this initial fragment.

The worker returns an integer expression $w(x)$ and a quantifier-free guard $C(x)$ such that

$$C(x) \iff \exists y \in \mathbb{Z}, \Phi(x, y), \quad C(x) \implies \Phi(x, w(x)). \quad (8)$$

The expression uses affine operations, positive-denominator floor division, remainder, minimum, and maximum. It is a single program for all parameters, not a table of unrelated witnesses for sampled inputs.

This directly extends the useful area between Forge’s integer-lattice worker and its finite-residue witness tables. A finite table can cost proportional to a modulus whose binary representation is short. The construction here uses the arithmetic structure of the congruences instead.

5.2 Normalize arbitrary fixed coefficients in congruences

For one congruence

$$cy \equiv \delta \pmod{m},$$

let $g = \gcd(c, m) > 0$ and choose integers u, v with $uc + vm = g$. A solution exists only if $g \mid \delta$. Under that condition the congruence is equivalent to

$$y \equiv u \frac{\delta}{g} \pmod{m/g}. \quad (9)$$

The quotient is an exact integer quotient under the guard. In the program it is computed with floor division, but the divisibility guard is never discarded. When $c = 0$, the normalization reduces to the guard $m \mid \delta$ and a modulus of one, imposing no restriction on y .

The certificate does not ask the checker to run the extended Euclidean algorithm. It supplies (g, u, v) , and the checker verifies

$$g > 0, \quad g \mid c, \quad g \mid m, \quad uc + vm = g.$$

These conditions imply that g is the positive gcd: every common divisor divides the displayed integer combination, while g itself is a common divisor.

5.3 Merge congruences with non-coprime moduli

Suppose the current system has been reduced to $y \equiv r \pmod{M}$, and the next normalized congruence is $y \equiv s \pmod{N}$, with $M, N > 0$. Put

$$h = \gcd(M, N), \quad pM + qN = h.$$

Compatibility requires $h \mid (s - r)$. Under that guard, set

$$k = \left(p \frac{s - r}{h} \right) \pmod{N/h}, \quad (10)$$

$$\begin{aligned} M' &= M(N/h) = \text{lcm}(M, N), \\ r' &= (r + Mk) \pmod{M'}. \end{aligned} \quad (11)$$

The two congruences are equivalent to $y \equiv r' \pmod{M'}$. In the total program, the displayed quotient $(s - r)/h$ is floor division; compatibility makes it exact. The residue expression therefore remains defined even when the compatibility guard is false.

Lemma 5.1 (Guarded CRT merge). *For the definitions above,*

$$(y \equiv r \pmod{M} \wedge y \equiv s \pmod{N}) \iff (h \mid (s - r) \wedge y \equiv r' \pmod{M'}).$$

Proof. If both congruences hold, writing $y - r = M\ell$ and $y - s = Nj$ shows $s - r = M\ell - Nj$, hence compatibility. Dividing by h , the equation for ℓ modulo N/h has inverse p for M/h , because $p(M/h) + q(N/h) = 1$. Therefore $\ell \equiv k \pmod{N/h}$, which is equivalent to $y \equiv r + Mk \pmod{M'}$. Conversely, compatibility and that last congruence give the first congruence directly and the second by substituting (10). Reducing $r + Mk$ modulo M' does not change its class. $\square$

Start with residue $r = 0$ and period $M = 1$, then merge every congruence. This produces a fixed final period and a symbolic residue program $r(x)$, together with divisibility guards. The moduli need not be prime, pairwise coprime, or small. Every normalized modulus and every merge uses its own checked Bezout receipt.

5.4 Convert inequalities into an interval

For $ay \leq \beta$:

$$\begin{aligned} a > 0 &\Rightarrow y \leq \lfloor \beta/a \rfloor, && \text{an upper bound,} \\ a = -c < 0 &\Rightarrow y \geq -\lfloor \beta/c \rfloor, && \text{a lower bound,} \\ a = 0 &\Rightarrow 0 \leq \beta, && \text{a parameter guard.} \end{aligned}$$

The negative-coefficient case is easy to get wrong. It uses $\lceil -\beta/c \rceil = -\lfloor \beta/c \rfloor$, with $c > 0$. Truncation toward zero would give incorrect answers for negative parameters.

Let $L(x)$ be the maximum of the lower bounds when any exist, and $U(x)$ the minimum of the upper bounds when any exist. Do not invent sentinel values for missing endpoints: the problem is over unbounded integers, not a machine range.

5.5 Choose the nearest congruent point

After normalization, all solutions to the congruences have the form $r + Mz$. If a lower bound exists, define

$$w = L + ((r - L) \bmod M). \quad (12)$$

This is the smallest integer at least L in the required residue class. If an upper bound also exists, add the final guard $w \leq U$.

If there is no lower bound but there is an upper bound, define instead

$$w = U - ((U - r) \bmod M). \quad (13)$$

This is the largest congruent integer at most U , so no extra interval feasibility guard is required. With neither bound, use $w = r$.

Theorem 5.2 (Constructive projection). *The conjunction of the normalization guards, CRT compatibility guards, zero-coefficient inequality guards, and the optional $w \leq U$ guard satisfies (8), with the witness defined above.*

Proof. The normalization and CRT lemmas establish an equivalence between the original congruences and their guards together with $y \equiv r \pmod{M}$. The inequality transformations are equivalences over integers, including the zero-coefficient cases. It remains to intersect one arithmetic progression with one possibly unbounded interval.

With a lower endpoint, (12) is congruent to r and lies in $[L, L + M)$. Any other congruent integer at least L differs from it by a nonnegative multiple of M . Thus an upper endpoint can be satisfied precisely when $w \leq U$. With only an upper endpoint, (13) always provides a suitable congruent point. With no endpoints, r itself suffices. These arguments prove both feasibility equivalence and correctness of the constructed witness, for every parameter valuation. $\square$

5.6 A witness program, not an exponentially large normal form

The implementation stores expressions as a hash-consed directed acyclic graph. A node refers only to earlier nodes and is one of:

<code>const(z)</code>	<code>var(i)</code>	<code>scale(z, node)</code>
<code>add(a, b)</code>	<code>div(a, d)</code>	<code>mod(a, d) // d > 0</code>
<code>max(a, b)</code>	<code>min(a, b)</code>	

Guards are divisibility tests or comparisons of node values. There is no floating point operation, no variable denominator, and no implicit conversion to machine integers. Shared affine expressions and intermediate residues need not be copied into a large tree-shaped term.

For fixed parameter arity, program-node count grows linearly with the number of constraints, apart from sharing effects. More explicitly, assembling affine right sides costs $O((p + q)(n + 1))$ nodes for p inequalities and q congruences. Integer coefficient bit lengths and the cost of division still matter. The final period can be a product of many moduli, but its bit length is at most the sum of their bit lengths before normalization. No loop runs once per residue class.

Example 5.3 (A 60-bit period in 42 nodes). The recorded large case has

$$m_1 = 1,000,000,007, \quad m_2 = 1,000,000,009,$$

$$M = m_1 m_2 = 1,000,000,016,000,000,063,$$

and constraints

$$x \leq y \leq x + M - 1, \quad y \equiv 2x + 1 \pmod{m_1}, \quad y \equiv 5x - 3 \pmod{m_2}.$$

The generated program has 42 nodes. The final interval has exactly one representative of every residue class, so the theorem establishes feasibility for all x . At $x = 0$ the generated witness is 2,000,000,015. The experiment also checks the program concretely at $x = \pm 10^{100}$. Those large values are implementation tests; the universal claim follows from the construction theorem, not from their size or number.

A residue table for this combined period is not a sensible representation. This is a representation-size contrast, not a measured speed comparison with Forge’s existing residue implementation.

5.7 How to discharge a quantified Lean goal

For a goal

$$\forall x, \quad D(x) \longrightarrow \exists y : \mathbb{Z}, \Phi(x, y),$$

introduce x and the domain assumptions, generate (C, w) , and create the remaining obligation $D(x) \Rightarrow C(x)$. Once this is proved, the generic projection theorem supplies the existential with witness $w(x)$. A local arithmetic solver can attempt the guard, while another worker may supply a missing congruence or bound. There is no reason to enumerate witnesses after the exact projection has already constructed one.

Conversely, a valuation of x violating C is useful only after checking the domain D and replaying the projection equivalence. A raw solver model for an unrelated abstraction is not evidence that the original universally quantified goal is false.

For natural-number output, first add $0 \leq y$ and prove that the selected integer can be transported to `Nat`. For rational or real parameters, this integer theorem is not the correct source semantics. Existential quantifier order also matters: a witness may depend only on parameters in scope before its binder.

5.8 The remaining verification work

The Python checker validates the Bezout receipts and deterministically reconstructs the witness program and guards from the input. It then compares this reconstruction with the supplied program. Search and checking share that assembler. Thus replay detects modified receipts or programs but cannot, by itself, detect an arithmetic mistake common to both uses of the assembler. The direct constraint evaluator and exhaustive concrete oracle in the test suite address that risk by a different route; they do not amount to a formal proof.

The Lean implementation should prove normalization, guarded CRT, bound conversion, and nearest-congruent-point lemmas once. Certificate elaboration then composes these lemmas, with positive-divisor facts explicit. A reflected program interpreter is an optional later optimization; an initial version can emit a shared `let`-term and ordinary proof applications. `omega` is an appropriate solver to try on residual linear integer obligations, but acceptance of this exact generated syntax must be measured rather than assumed. The mathematical construction does not require any SMT solver.

5.9 Limits that must survive the interface

Variable coefficients such as $xy \leq z$, variable moduli, nonlinear parameter right sides in the current input grammar, general Boolean combinations, and several mutually constrained existential outputs are outside the implemented fragment. Disequalities can be handled by explicit branching in a future layer, but they are not silently dropped. Repeatedly projecting several variables may cause large piecewise expressions and requires a separate supported-language argument after each elimination. A one-output completeness theorem must not be advertised as complete arbitrary Presburger synthesis.

6 Proof-directed higher-order sketch decomposition

Status. Integration design, not an implemented synthesizer. The archive includes a hand-written, uncompiled `List/Int` fold specimen. It does not close Leant’s original Church-encoded indexing query.

6.1 The specific obstruction in Leant and Djex

The inspected Leant report describes an unusually useful diagnostic. Under the original bounded Haskell Exference indexing query, all 256 inspected candidates were behaviorally falsified. Yet a suitable integer-case step synthesized in isolation at candidate 5, and a corresponding step in a larger ambient context appeared at candidate 219. Function-valued folds also appeared separately. These observations locate a composition and ordering problem, not simply a missing source-language construct [7].

The report explicitly warns that candidate 219 is not the internal ordinal of that step in the original fold search. Partial terms, pending goals, usage history, and context differ. Accordingly, the proposal here is not to infer a magic ranking penalty from those two numbers. It is to make a proof schema expose smaller, semantically specified synthesis problems before complete candidate terms are enumerated.

6.2 The proof rule should determine the holes

For a right fold, choose a carrier type B and a relation

$$\mathcal{R} : \text{List}(A) \rightarrow B \rightarrow \text{Prop}.$$

The two proof obligations are

$$\mathcal{R}([], b_0), \tag{14}$$

$$\forall a, \ell, b, \quad \mathcal{R}(\ell, b) \implies \mathcal{R}(a :: \ell, \text{step}(a, b)). \tag{15}$$

They imply

$$\forall \ell, \quad \mathcal{R}(\ell, \text{foldr}(\text{step}, b_0, \ell)) \tag{16}$$

by structural induction. Forge can register this as a reusable proof theorem. The synthesis task is then to construct b_0 and `step` satisfying these contracts, not an entire recursive program from a bare outer type.

The relation is the key. Merely choosing the fold combinator and leaving its holes constrained only by types reproduces the original ambiguity. Conversely, choosing an arbitrary expressive relation and asking a solver to discover it is not yet an algorithm. A first implementation should use an explicit inventory of specification templates: equality with a reference fold, a size equation, a pointwise relation for function carriers, a predicate preserved by constructors, or a packet of arithmetic invariants discovered by Section 3.

6.3 Indexing explains why the carrier must be a function

Let $d : A$ be a default. Define a total reference operation on ordinary lists and mathematical integers by

$$\begin{aligned} \text{at}(d, [], i) &= d, \\ \text{at}(d, a :: \ell, i) &= \begin{cases} d, & i < 0, \\ a, & i = 0, \\ \text{at}(d, \ell, i - 1), & i > 0. \end{cases} \end{aligned}$$

The source list is the structural argument, while the index changes during processing. Lambda-lifting the index into the fold result suggests

$$B = \mathbb{Z} \rightarrow A, \quad \mathcal{R}(\ell, k) \equiv \forall i, k(i) = \text{at}(d, \ell, i).$$

The base hole has the contract $\forall i, b_0(i) = d$. The step hole has the contract

$$\text{step}(a, k)(i) = \begin{cases} d, & i < 0, \\ a, & i = 0, \\ k(i - 1), & i > 0. \end{cases} \quad (17)$$

The simple solution is therefore

```
base = fun i => default
step = fun head continuation i =>
  if i < 0 then default
  else if i = 0 then head
  else continuation (i - 1)
answer = foldr step base xs index
```

When an approved integer-case primitive already has exactly this contract, as in the Leant diagnostic, it should be tried as a whole component before synthesizing its branch structure. This is a concrete use for Djex: solve a small typed component problem under an explicit semantic contract, then return the candidate through its existing source-typed boundary. Forge supplies the fold theorem and the proof obligations that the type-directed search alone does not express.

The carrier heuristic is also specific: when a non-structural argument changes in a recursive specification and the result type is R , try the corresponding function type $I \rightarrow R$ as a fold carrier. For several changing arguments, try a curried or product carrier with the original dependency order. This is an OR alternative, not a theorem that all such functions are folds.

6.4 Separate executable context from specification context

The step program in (17) needs d, a, k, i , but not the original whole list or the original outer index. The proof of its contract may still quantify over the tail list ℓ and use $\mathcal{R}(\ell, k)$. Thus there are two contexts:

$$\Gamma_{\text{code}} = (d, a, k, i), \quad \Gamma_{\text{proof}} = \Gamma_{\text{code}}, \ell, h : \mathcal{R}(\ell, k).$$

A ghost variable may occur in the specification without being available as a program component. Erasing it from the proof context would lose the induction hypothesis; making it available as an executable value could let the synthesizer bypass the intended fold by calling a reference function on the tail.

Context projection therefore operates on the executable inventory and closes under all type dependencies. If a retained variable's type mentions another local, that dependency is retained or abstracted explicitly. The full source context remains available to the proof checker. The small-context search is a particular grammar restriction, not a globally sound reason to prune every larger-context search branch. A separate broader alternative can remain queued.

This proposal sharpens the diagnosed context effect without claiming that the specific Leant miss has already been traced to one variable or one penalty. It also makes an implementation experiment precise: retain the original outer query, but replace the internal monolithic branch with the registered fold schema and separately constrained holes.

6.5 A concrete search protocol

```

prepare_specification(goal):
  recover source function, recursive argument, result, and laws
  retrieve approved recursor/fold schemas by input and result shape
  propose carriers by dependency-preserving lambda lifting
  for each schema and carrier:
    instantiate a registered relation template
    derive base and step contracts by applying the schema theorem
    compute executable inventories for the holes
    retain ghost variables and assumptions in proof contexts
    enqueue this plan as an alternative

solve_plan(plan):
  request typed candidates for each hole from Djex or native search
  reject concrete counterexamples where evaluation is available
  prove the candidate's whole hole contract in Lean
  retain a universally quantified hole theorem, not a sample verdict
  apply the schema theorem to assemble the original proof
  replay at the exact original goal and source inventory

```

The initial component grammar should contain local values, constructors, projections, applications of approved providers, and the selected integer-case primitive. For first-order arithmetic holes, the constructive projection worker can replace candidate enumeration. More general finite arithmetic grammars can be sent to cvc5's SyGuS interface as proposal problems, followed by Lean proof of the returned candidate's contract. The documented cvc5 API provides `synthFun`, `addSygusInvConstraint`, and synthesis-result extraction; no cvc5 adapter was executed in this work [15].

The relation to refinement-type synthesis is direct: semantic types decompose the specification before full programs are built. Synquid is a primary example of this approach. TYGAR provides a complementary example of using an abstract type graph and refinement to guide component composition [13, 14]. The proposed Forge specialization is to use already proved Lean recursor laws as the decomposition authority and to make the resulting hole theorems reusable in the original proof. It is not a new claim of decidability for dependent or impredicative synthesis.

6.6 Church encodings require an explicit representation law

The original diagnostic uses a Church-encoded list, not Lean's inductive `List`. A theorem about `List.foldr` alone does not close that query. To transfer the sketch, require a relation such as

$$\text{Rep}(c, \ell) \implies c \ B \ \text{step} \ b_0 = \text{foldr}(\text{step}, b_0, \ell),$$

at the exact carrier and universe selection in use. For an explicit encoding of an ordinary list, this is a straightforward fold law. For an arbitrary polymorphic value, it must not be inferred merely from its displayed type. Parametricity, representability, totality, and available eliminations are substantive assumptions, especially across Haskell and Lean.

The source Haskell `Int` is not automatically the same object as Lean's unbounded `Int`. The actual branch primitive, source binder order, polymorphic selections, and dictionary payloads must be retained. The archive's `IndexingSketch.lean` intentionally proves only a proposed equation between two ordinary-list definitions at mathematical integers. It is hand-written and uncompiled, and it is not substituted for acceptance of the original query.

6.7 What should be measured in the real integration

Record the number of schema alternatives, candidate holes, full contract checks, reusable hole theorems, and reconstructed whole programs. Compare the same outer query, source inventory, budgets, and acceptance predicate. An isolated hole success is diagnostic evidence, not a solved original benchmark. Preserve a negative control that actually reaches predicate checking, rather than calling a zero-candidate timeout a successful rejection.

The important hypothesis is that proof-directed decomposition reduces wasted whole-program verification and context-induced branching. This hypothesis is plausible from the inspected diagnostic but remains unmeasured here. It should be tested before changing Djex’s global search penalties or claiming a broader Church-encoding milestone.

7 A cross-worker proof, with a precise division of labor

Consider a full binary tree t , with $L = L(t)$ leaves and $N = N(t)$ internal nodes. A goal combining structural and arithmetic reasoning is

$$\exists y \in \mathbb{Z}, \quad N \leq y < N + 15 \wedge 2y \equiv L - N + 1 \pmod{6} \wedge y \equiv N \pmod{5}. \quad (18)$$

Here natural counts are explicitly cast to integers.

The finite-algebra worker, using counts modulo six, proves $L - N \equiv 1 \pmod{6}$ for all trees. Hence $L - N + 1 \equiv 2 \pmod{6}$, supplying the divisibility guard required to normalize the first arithmetic congruence. The integer worker reduces it to $y \equiv 1 \pmod{3}$, merges it with $y \equiv N \pmod{5}$, and chooses the first such integer at least N . Its period is 15, so the chosen value is strictly below $N + 15$.

For example, with a normalized residue representative

$$r(N) = (1 + 3(2(N - 1) \bmod 5)) \bmod 15,$$

one possible displayed witness is

$$w(N) = N + ((r(N) - N) \bmod 15).$$

The factor 2 is the inverse of 3 modulo 5. This is a hand-simplified explanatory witness; the generic prototype retains its own instruction graph rather than claiming that the simplification was discovered automatically.

The structural component supplies a proved property of all trees. The arithmetic component supplies one uniform witness and proves its constraints. Neither worker invents assumptions to make the other succeed. The existing Forge controller only needs to route the missing divisibility demand to the finite summary worker and then introduce the resulting proof. This composition is a specified end-to-end target, not a Lean run in the archive.

8 Lean implementation plan and concrete dependencies

8.1 Use the existing outer boundary

Forge already recommends starting with ordinary Lean goals and feeding checked facts to a fresh local `grind` run. The current Lean documentation describes `grind` as cooperating local solvers with proof construction; these workers should augment that machinery, not reimplement it [4, 9]. No new shared congruence-closure database is required for the first version.

Use four focused modules behind the existing worker interface:

```
Forge.Worker.InvariantSpace    // constant and polynomial multipliers
Forge.Worker.FiniteCover      // cover or concrete tree
Forge.Worker.IntProjection     // guard and uniform witness
Forge.Planner.ContractSketch  // proof-directed hole decomposition
```

These are proposed module names. None is present as an implemented Lean worker in this archive.

8.2 A minimal certificate-to-proof path for each worker

Invariant packets. Reification returns a polynomial syntax for each source expression together with a theorem relating its evaluation to that expression. A sparse normalizer proves that equality of canonical coefficient maps implies equality of evaluations. The checker validates initial substitutions and multiplier identities, then instantiates a generic reachability-induction theorem. To derive a particular goal equality from the packet, supply and check its coordinates as a constant or polynomial combination of the generators. A packet is useful only after this last target connection is made.

For rational coefficients interpreted in an ordered field or another characteristic-zero field, use an explicit coefficient homomorphism. For integer source states, clearing denominators is often simpler than transporting many rational divisions. Neither method licenses dropping a source cast or treating natural subtraction as integer subtraction. The reifier must reject unsupported operations rather than silently interpreting them as polynomial operations.

Finite covers. A positive certificate is a finite set of states; a reflected checker establishes the constructor closure and good-state inclusion. A generic structural theorem then yields the source property using the registered summary laws. A negative certificate is a constructor DAG, evaluated and reconstructed as a concrete source term. The two result forms must have different proof goals.

The included `CoreSoundness.lean` states the generic reachability and tree cover theorems in a small core-only form. It does not include the reflected finite checker or source reifier. Finite verification can begin with ordinary `decide` on small explicit instances; larger arrays may justify a dedicated reflected checker with a proved interpreter. An unchecked native computation must not be substituted for that theorem.

Integer projection. The first implementation can elaborate each receipt by theorem application rather than formalizing the entire search algorithm. Prove the congruence normalization, CRT merge, floor-bound, and nearest-point lemmas once. Decode the program as a shared expression with positive constant divisors. Return both an equivalence proof for the feasibility guard and a proof for the selected witness under that guard. Keeping only the second loses useful negative and routing information; keeping only the first loses constructive synthesis.

Contract sketches. A registered schema supplies the proof of composition. Hole candidates are elaborated at exact source types and checked against their full contracts. Assemble the final expression and theorem only after all contracts are proved. The source-typed Djex boundary is a sensible candidate-provider API; the behavioral observations in its diagnostic reports are filters, not replacements for these universal contracts [8, 6].

8.3 Library choices, with their roles kept narrow

Component	Concrete dependency	Role and boundary
Local proof closure	Lean <code>grind</code> , <code>omega</code> ; Mathlib <code>ring</code>	Close ordinary proof obligations after explicit side conditions are supplied
Polynomial semantics	Mathlib <code>MvPolynomial</code> , especially evaluation homomorphisms	Source denotation and correctness lemmas; not external search authority
Finite certificates	Lean/Mathlib <code>Array</code> , <code>Fin</code> , <code>Fintype</code> , <code>Finset</code>	Enumerate finite states and prove a checker correct
Exact proposer algebra	<code>fractions.Fraction</code> now; optional <code>python-flint.fmpq_mat</code>	Nullspaces and rational solving outside the kernel
Integer proposer	Extended Euclid plus the included DAG assembler	Deterministic construction; every arithmetic lemma reconstructed in Lean
Typed search	component: Djex’s checked public synthesis boundary	Supply source-owned hole candidates, preserving polymorphism and providers
Optional search	grammar: <code>cvc5</code> SyGuS	Propose first-order arithmetic candidates; replay their contracts in Lean

The table names implementation components, not a tested dependency matrix. Mathlib’s current documentation uses the path `Mathlib/Algebra/MvPolynomial/Eval.lean`; names and imports must be checked against the project’s pinned Mathlib revision, not inferred from a historical `Data/MvPolynomial` path [17]. The pinned Forge baseline reports Lean 4.34.0 and a specific Mathlib commit; the Python prototypes do not depend on that installation and do not establish adapter compatibility with it.

8.4 Mathematical contracts for the native interface

The prepared problem should describe which interpretation theorem the worker is allowed to produce. A useful separation is:

```

InvariantProblem -> packet identities -> reachable-state theorem
FiniteProblem    -> cover proof | concrete-tree proof
IntegerProblem   -> (guard equivalence, witness correctness)
HoleProblem      -> candidate term with full contract proof

```

A closed packet about an auxiliary transition system is not yet a proof of an unrelated target. A finite cover for an abstraction is not a concrete counterexample. A projection guard is not an existential witness until its premise is proved. These are worker-specific proof obligations beyond the identity and scope discipline already specified by Forge.

For replay, retain the re-checkable problem and certificate, not a producer’s cached verdict. The existing Forge contracts for exact source identity, rollback, and proof acceptance remain applicable; this article does not replace them with a second protocol.

8.5 One particularly useful proof-reuse boundary

Compile a successful search into a theorem specialized to its transition maps, finite algebra, or arithmetic coefficients. The general checker theorem can be shared, while the certificate itself becomes finite checked data. Recompile then need not rerun Gaussian elimination, reachability search, or a synthesis solver. This applies equally to a reusable hole theorem from the sketch planner.

Avoid serializing expanded polynomial normal forms into a huge source script when a coefficient vector and a generic normalization proof suffice. Conversely, for a small example, a short ordinary proof may be simpler and faster than a new reflected interpreter. Measure both source size and kernel-checking cost at the actual target before choosing the permanent representation.

9 Executed experiments and what they establish

9.1 Environment and units

The recorded run used Python 3.13.5 on Linux x86-64. The command line used `python -S`, disabling site-package initialization. The implementations import only the standard library. Neither `lean` nor `lake` was available, and the Lean status file records NOT RUN rather than a successful elaboration.

There is one new experiment ledger, separate from Forge’s nine historical runs. A certificate, a unit-test method, a parameter valuation, a state, and a tuple evaluation are different units. The tables keep them separate. In particular, these records must not be appended to a historical solved-goal total.

Lane	Stored certificates	Replay accepted	Contents
Constant multipliers	51	51	Cyclic packet, recurrence, 48 affine cases, one expected miss
Polynomial multipliers	14	14	Squaring, union, parabola, false-line control, powers 2 through 9
Finite algebra	136	136	Count covers, mirror examples, and random exact algebras
Integer projection	221	221	220 random symbolic systems and one large-period system

There are 422 stored certificates in this one corpus. Some are concrete counterexamples and some contain an empty invariant packet. The count is not 422 proved Lean theorems, and it is not a solved-goal benchmark.

9.2 Invariant tests and ablations

The cyclic-differences case returns a dimension-two invariant space while its conserved-linear subspace has dimension zero. The nonlinear squaring-diagonal case returns no generator with multiplier degree zero and one generator with multiplier degree one. The union-of-axes case changes from zero to one generator when the multiplier bound rises from one to two. The false-line control confirms that an initially true relation is not retained merely because it fits the initial parameterization.

The 48 affine-diagonal instances have three state variables, one to three transitions, and alternating template degrees one and two. Random affine maps are constructed to preserve the diagonal. The computed dimensions are two and seven, respectively. These engineered cases exercise simultaneous invariants, multiple transitions, and coefficient handling; they are not a representative sample of all programs.

Example	d	e	Recorded result
Cyclic differences	1	0	Two generators; no nonzero conserved linear generator
Sums of squares	3	0	One generator after dimensions 9, 5, 3, 2, 1, 1
Squaring, initial $x = y$	1	0	Empty packet: expected certificate-language miss
Squaring, initial $x = y$	1	1	$y - x$ with multiplier $x + y$
Squaring, initial $xy = 0$	2	1	Empty packet: multiplier bound too small
Squaring, initial $xy = 0$	2	2	xy with multiplier xy
Squaring, initial $y = x^2$	2	2	$y - x^2$ with multiplier $y + x^2$

Displayed generators can differ by sign or rational scaling from the stored row-reduced basis. The actual basis and exact multiplier matrices remain in the certificate file. The table explains their mathematics rather than rewriting the recorded evidence.

9.3 Finite-algebra differential checks

The count family covers moduli 1 through 12. Each case returns a cover with exactly m states in a product of size m^2 . The mirror family has 24 magma instances: 12 commutative cases return covers and 12 deliberately noncommutative cases return counterexample trees. The 100 random finite algebras return 33 covers and 67 counterexamples, agreeing with a separately written full-closure oracle.

The oracle computes all reachable states even when the production proposer has already found a bad state. Thus the comparison checks the verdict against the whole exact algebra, not only against the first explored tree. The positive certificate checker separately enumerates all constructor tuples in the returned cover. The negative checker independently evaluates its acyclic tree. Both still share input-table semantics with the search, a much smaller shared surface than the worklist algorithm but not a formal verification of it.

9.4 Integer differential checks

The 220 random symbolic systems use one or two parameters, fixed output coefficients in $[-4, 4]$, moduli from 1 through 8, up to four inequalities, and up to three congruences. At each parameter valuation from $\{-4, -1, 0, 2, 5\}^n$, the generated feasibility guard is compared with a complete finite enumeration for that *concrete* problem.

Let B be one plus the largest absolute right-side value, and let P be the lcm of the input moduli, or one if there are none. Every finite bound endpoint lies in $[-B, B]$. The congruence solution set is periodic with period dividing P . Therefore any satisfiable interval, including a one-sided or fully unbounded interval, has a witness in $[-B - P, B + P]$. Enumerating that range is a complete concrete oracle for the generated test inputs, not merely an arbitrary small box.

Across 3,400 valuations, the guard reports 1,368 feasible and 2,032 infeasible instances, all agreeing with the oracle. Every feasible valuation also checks the selected witness directly against the original input constraints. The large-period case is tested separately at five values, including $\pm 10^{100}$; it is not subjected to impossible modulus-sized enumeration.

9.5 Corruption and implementation controls

All 21 unit-test methods pass. Their internal cases include malformed rational coefficients, matrix and multiplier corruption, wrong initial behavior, changed problem subjects, an insufficient multiplier budget, an empty or unsafe cover, a self-referential tree, a good state falsely designated as a counterexample, changed Bezout coefficients, deleted feasibility guards, a changed witness

node, and floating-point or Boolean values where exact integers are required. There are 19 explicit certificate-mutation cases in those methods, in addition to malformed-polynomial and semantic differential cases.

The replay script disables synthesis, row reduction, nullspace, coordinate search, and extended-gcd construction by replacing their entry points with exceptions before checking the stored corpus. Every certificate still replays. This demonstrates separation from those search routines; it does not make shared polynomial arithmetic or the integer assembler formally verified.

9.6 Timings are diagnostic only

The raw ledger includes one observed duration per lane. In the recorded run, constant-multiplier experiments took approximately 0.613 seconds, polynomial multipliers 0.037 seconds, finite algebras 0.023 seconds, and integer projections 0.260 seconds. These figures include that lane’s construction, checks, and specified oracles, but exclude process startup and final JSON serialization. The lanes perform different work, and there are no repeated trials, confidence intervals, or matched Lean baselines. The figures are useful for detecting gross reproduction problems, not for claiming comparative performance.

10 Implementation order and acceptance gates

10.1 First: one source theorem per certificate family

Begin by elaborating the supplied generic Lean specimens under the pinned project and implementing one actual source bridge per family. A good minimum set is the cyclic packet, a binary-tree count congruence, and a guarded non-coprime integer witness. Require a complete proof of the exact source goal, with recorded axiom dependencies, and then rerun it without the proposer. This establishes something the Python corpus cannot: a certificate really can cross the source-language boundary into a Lean proof.

10.2 Second: native certificate checkers, not native searches

Port the deterministic checker first. The external Python proposer can continue to supply exact finite data. For finite covers, prove a table-closure checker sound. For integer projection, compose the four generic arithmetic lemmas. For invariant packets, finish polynomial reification and identity checking. Only then optimize or reimplement the external searches in Lean, FLINT, or another language if measurements justify it.

The polynomial multiplier algorithm is mathematically the most substantial addition, but its source polynomial bridge is also the most delicate. The finite cover checker is a useful smaller first end-to-end milestone. This is an engineering order, not a ranking of the algorithms’ eventual importance.

10.3 Third: benchmark the actual new capability

Build source-goal families that isolate the extension: simultaneous invariants without conserved generators; invariant ideals requiring positive multiplier degree; arbitrarily deep trees summarized by a finite algebra; and integer witnesses with large symbolic periods. Include true goals outside the supported fragments and false goals whose counterexamples are checked.

Compare against **grind** with the same definitions and permitted theorem inventory, a reasonable hand-orchestrated baseline, and relevant specialized tactics. Report separately preparation, search, certificate size, reconstruction, kernel checking, and cached replay. Hold out theorem families

rather than only renaming variables, and prevent retrieval from importing the exact target lemma. None of those controlled Lean measurements is claimed by this article.

10.4 Fourth: test the original higher-order query unchanged

Integrate the contract-sketch branch into the actual Leant/Djex path and run the original indexing query with its original source type, provider inventory, integer semantics, observations, and resource limits. Retain exact branch traces showing carrier choice, hole inventories, and candidate order. The list-fold specimen and isolated step successes are useful controls; neither substitutes for this gate. The simultaneous two-universe query remains a separate target.

11 Assessment

The strongest addition is the bounded-multiplier fixed point. It provides a finite, exact way to synthesize entire invariant packets, rather than asking a bilinear solver to guess generators and their closure multipliers at once. Finite-algebra covers give another precise form of induction: a small closure certificate stands for all finite constructor trees. Constructive projection turns one-output integer feasibility into a compact program instead of a residue table. Contract-first sketches explain how these mathematical workers can cooperate with a sophisticated type-directed synthesizer without reducing the problem to a larger unstructured candidate stream.

The mathematical claims are deliberately local and useful. The invariant worker is complete for its bounded certificate space; affine transitions admit the stronger degree-bounded equality result. The finite worker decides its explicit finite algebra. The integer worker exactly projects its stated one-output conjunctive fragment. None is a complete general Lean prover, and none needs to be one to add a qualitatively different successful proof route.

The executable archive establishes that the core constructions work on the recorded cases, reject the specified corruptions, and replay without their search routines. The next decisive result is not a larger Python pass count: it is a checked source theorem from each family, followed by a fair measurement in the actual Forge environment. The article supplies the algorithms, certificate contracts, proofs, concrete implementation choices, and reproducible artifacts needed to take that step.

A Reproduction and source revisions

A.1 Archive layout

article/	this PDF, editable TeX, bibliography, generated result counts
prototype/	exact arithmetic, four algorithm modules, tests, run and replay
results/	certificates, structured ledger, test and replay transcripts
lean/	two uncompiled specimens, pinned toolchain, check script, status
docs/	source provenance and acceptance ledger
README.md	commands and scope
LICENSE	MIT-0 for the new supplementary material

The archive does not redistribute the original Forge article, the neighboring repositories, third-party papers, external library source, or font files.

A.2 Commands

From the extracted archive root:

```
cd prototype
python -S -m unittest -v test_prototypes
python -S verify.py
python -S run_experiments.py --output ../reproduced-results
```

The final command writes a new directory by default; it does not overwrite the recorded results. All these commands use the standard library only. Python 3.9 or later is the intended syntax/API baseline; the recorded execution used Python 3.13.5, not a cross-version CI matrix.

Build the article with ordinary pdfLaTeX:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
```

The document uses stock TeX packages, no shell escape, no bibliography tool, and no separately distributed system fonts.

Optional Lean elaboration, after installing the pinned toolchain:

```
python lean/check.py
```

This command writes `lean/status.json` with actual outcomes and exits unsuccessfully when the compiler is absent or a specimen fails. The producing run records absence of the Lean executable. The specimens are neither generated certificates nor an implementation of a tactic.

A.3 Pinned audit references

Forge

674521027d968d59f7b83220ed52304a85cb55e2. The read material comprises the README, the Leant/Djex review, the structural search chapter, the first 200 lines of the integration chapter, and the conclusion. A recursive repository tree was inspected and the additional keyword searches are recorded in `docs/provenance.json`.

Leant

6bf05ad78c467989e68290f2d08bbbed40802d485. The opening README material and the full dated indexing-composition diagnostic were read. That report identifies its Djex runtime/dependency as `e237e866`; this is not the same as the separately inspected Djex head.

Djex

e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. The opening 105 README lines and current branch identity were inspected. No Djex source modification or live synthesis run was performed.

Forge's reported target toolchain

Lean v4.34.0; Mathlib 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. These are repository baseline identities, not a claim that the supplementary Lean files compiled against them.

The repositories were read through the connected GitHub interface. No files, branches, issues, or pull requests in them were changed. The contribution is the standalone supplementary archive.

B Certificate schemas and small API examples

B.1 Exact polynomials

A polynomial is a list of terms

```
[exponent_vector, numerator, positive_denominator]
```

with nonnegative integer exponents. Rational coefficients are reduced, nonzero, and exact. The writer sorts exponents; the decoder rejects duplicates, noncanonical coefficients, incorrect arities, and invalid numeric types. Mathematical equality is polynomial equality, not equality of formatted text.

B.2 Invariant packets

A constant packet has schema `forge.pullback-space/1`, a complete problem, a generator list, and rational transition matrices. A polynomial packet has schema `forge.ideal-space/1`; its problem also includes the multiplier bound and its matrix entries are polynomials. The matrix convention is

$$p_i \circ T_a = \sum_j H_{aij} p_j.$$

The caller supplies the expected problem separately to the checker. The copy in the certificate is compared with that expected problem; it is not allowed to define the caller’s proof obligation by itself.

A minimal nonlinear example:

```
import exact as ex
import ideal_space as ideal

x, y = ex.var(2, 0), ex.var(2, 1)
certificate, statistics = ideal.synthesize(
    2, 1, 1,                                # variables, generator degree, multiplier degree
    [[x, x]],                               # initial diagonal, parameterized exactly
    [[ex.mul(x, x), ex.mul(y, y)]]          # squaring transition
)
assert ideal.check(certificate['problem'], certificate)
print(statistics)
```

An empty generator list is legal but only certifies a vacuous invariant packet. The source-level user interface should report that no useful packet was returned, not that the target is false.

B.3 Finite algebras

The problem records the number of states, each constructor’s name and arity, its row-major transition table, and the set of good states. A cover certificate lists states. A counterexample certificate lists constructor nodes with earlier child indices and a root index. Arity checks and the earlier-index discipline are necessary for interpreting the node list as a finite constructor tree.

```
import finite_cover as fc

problem = fc.binary_tree_counts(7)
certificate, statistics = fc.synthesize(problem)
assert certificate['kind'] == 'cover'
assert fc.check(problem, certificate)
```

The transition budget’s `unknown` output is not a certificate and is rejected by `check`. Table construction and parsing need additional production resource limits beyond this prototype’s evaluation counter.

B.4 Integer projection

An affine parameter expression is stored as $[b, a_1, \dots, a_n]$, denoting $b + \sum a_i x_i$. An inequality row represents $a y \leq \beta(x)$; a congruence row represents $a y \equiv \beta(x) \pmod{m}$. The certificate stores two Bezout receipts per congruence, one for coefficient normalization and one for its CRT merge.

```
import presburger as pb

problem = {
    'parameters': 1,
    'inequalities': [
        {'a': -1, 'rhs': [0, 0]},    # y >= 0
        {'a': 1, 'rhs': [10, 0]}    # y <= 10
    ],
    'congruences': [{'a': 2, 'rhs': [0, 1], 'm': 6}]
}
certificate = pb.synthesize(problem)
assert pb.check(problem, certificate)
feasible, witness = pb.evaluate_program(certificate['program'], [8])
assert feasible and pb.holds(problem, [8], witness)
```

Evaluation is intended for a program already accepted by the checker. The concrete evaluator does not itself revalidate an arbitrary untrusted instruction stream on every call.

C Acceptance ledger

Claim	Status	Evidence or remaining gate
Constant-packet construction and replay	Executed	51 stored certificates; exact identity checker
Polynomial-multiplier fixed point and replay	Executed	14 stored certificates; degree and identity checks
Finite covers and concrete counterexamples	Executed	136 certificates and a separate random closure oracle
One-output integer projection	Executed	221 certificates; 3,400 small-parameter differential valuations
Search-free replay	Executed	All 422 certificates pass after search entry points are disabled
Certificate corruption checks	Executed	19 explicit mutated certificates rejected within 21 test methods
Mathematical soundness of the formats	Proved in text	Induction, polynomial identities, and integer congruence lemmas
Correctness of the Python implementation	Tested, not formalized	Shared polynomial arithmetic and shared integer assembler remain
Lean specimens	Not run	Compiler absent; status recorded explicitly
Lean source reifiers and native checkers	Designed only	Must produce complete checked source theorems
Contract-directed Djex integration	Designed only	Must preserve and pass the original outer query
Cross-worker binary-tree witness example	Designed only	Mathematical derivation included; no end-to-end Lean run
Performance versus existing tactics	Not measured	Requires matched source goals, budgets, and premise inventories

References

- [1] VladimirReshetnikov/Forge. *README and repository status*. Revision 674521027d968d59f7b83220ed52304a85cb55e2, inspected 15 September 2026. Pinned repository source.
- [2] VladimirReshetnikov/Forge. *Structural search: changing the statement that is being proved*. Same revision; `article/sections/05-structural.tex`. Pinned repository source.
- [3] VladimirReshetnikov/Forge. *Ideas from Leant and Djex*. Same revision. Pinned repository source.
- [4] VladimirReshetnikov/Forge. *Lean integration: a conservative bridge before an invasive one*. Same revision; inspected opening integration sections. Pinned repository source.
- [5] VladimirReshetnikov/Forge. *Assessment and recommended starting point*. Same revision. Pinned repository source.
- [6] VladimirReshetnikov/Leant. *Leant—a Djex-based synthesis REPL for Lean 4*. Revision 6bf05ad78c467989e68290f2d08bbbed40802d485; opening README material, inspected 15 September 2026. Pinned repository source.
- [7] VladimirReshetnikov/Leant. *Indexing composition after native kind adoption*. 14 September 2026; same revision as above. Pinned repository source.
- [8] VladimirReshetnikov/Djex. *Djex*. Opening README material and branch identity, revision e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5, inspected 15 September 2026. Pinned repository source.
- [9] Lean language reference. *The grind tactic*. Official documentation, accessed 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] Michael Karr. *Affine relationships among variables of a program*. Acta Informatica 6, 133–151, 1976. DOI: 10.1007/BF00268497. <https://doi.org/10.1007/BF00268497>.
- [11] Ehud Hrushovski, Joël Ouaknine, Amaury Pouly, and James Worrell. *Polynomial Invariants for Affine Programs*. LICS 2018. <https://arxiv.org/abs/1802.01810>.
- [12] Peter Lammich. *Tree Automata*. Archive of Formal Proofs, 25 November 2009; current entry accessed 15 September 2026. <https://isa-afp.org/entries/Tree-Automata.html>.
- [13] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016. DOI: 10.1145/2908080.2908093. <https://cseweb.ucsd.edu/~npolikarpova/publications/pldi16.pdf>.
- [14] Zheng Guo, Michael James, David Justo, Jiaxiao Zhou, Ziteng Wang, Ranjit Jhala, and Nadia Polikarpova. *Program Synthesis by Type-Guided Abstraction Refinement*. arXiv:1911.04091, 2019. <https://arxiv.org/abs/1911.04091>.
- [15] cvc5 developers. *SyGuS: Invariants*. Official API examples and documentation, accessed 15 September 2026. <https://cvc5.github.io/docs/latest/examples/sygus-inv.html>.
- [16] python-flint developers. *fmpq_mat—matrices over rational numbers*. Official documentation, accessed 15 September 2026. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html.
- [17] Mathlib contributors. *Mathlib.Algebra.MvPolynomial.Eval*. Generated API documentation, accessed 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/MvPolynomial/Eval.html.